Chain of Responsibility

Chain · Route support tickets billing→tech→general · Build with Waqar

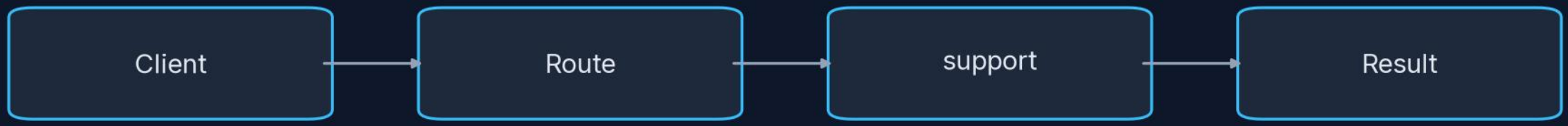
Today's Hook

- Today's force: Route support tickets billing→tech→general
- We earn Chain by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Handler, BillingHandler, TechHandler, GeneralHandler

The Problem

- Scenario: Route support tickets billing→tech→general
- Constraints: solve «Route support tickets billing→tech→general» with Chain — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Route support tickets billing→tech→general
- Success looks like: billing-desk

Structure



Chain: Route support tickets billing→tech→general

Real-World Scenario

- Running example: Route support tickets billing→tech→general
- Key types: Handler, BillingHandler, TechHandler, GeneralHandler
- Happy-path stdout cue: billing-desk
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.chain.problem;

import java.util.*;

public final class NaiveRouter {
  public static String route(String type){
    if("billing".equals(type)) return "billing-desk";
    if("tech".equals(type)) return "tech-desk";
    return "general-desk";
  }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Route support tickets billing→tech→general
- Without Chain, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.chain.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;

abstract class Handler {
  private Handler next;
  Handler link(Handler n){ next=n; return n; }
  final String handle(String type){
    String r = doHandle(type);
    if (r!=null) return r;
    return next==null? "unhandled": next.handle(type);
  }
  abstract String doHandle(String type);
}
final class BillingHandler extends Handler { String doHandle(String t){ return "billing".equals(t)? "billing-desk": null; } }
final class TechHandler extends Handler { String doHandle(String t){ return "tech".equals(t)? "tech-desk": null; } }
final class GeneralHandler extends Handler { String doHandle(String t){ return "general-desk"; } }
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.chain.problem;

import java.util.*;
import java.util.concurrent.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Chain: support ticket routing ===");
    System.out.println("--- naive ---");
System.out.println(NaiveRouter.route("billing"));
System.out.println("--- chain ---");
var h=new BillingHandler(); h.link(new TechHandler()).link(new GeneralHandler());
System.out.println(h.handle("billing"));
System.out.println(h.handle("tech"));
System.out.println(h.handle("other"));
  }
}
```

Walkthrough Output

```
=== Chain: support ticket routing ===
--- naive ---
billing-desk
--- chain ---
billing-desk
tech-desk
general-desk
```

Step-by-Step Trace

- Match stdout to stages — look for: billing-desk
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 15-chain-of-responsibility ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Chain
// Prefer records + sealed where they shrink the demo.
// Problem under test: Route support tickets billing→tech→general
```

Trade-offs

- Use Chain when «Route support tickets billing→tech→general» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Chain in product language.

Related Patterns

- Name the pattern usually confused with Chain and separate the forces.
- Sketch the roles for «Route support tickets billing→tech→general» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Route support tickets billing→tech→general
- Naive pain → Chain boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.